

Service, DTO ve Dependency Injection

Controller'ı hafiflet; iş kurallarını service'e taşı, dışarıdan gelen veriyi DTO ile sınırla ve bağımlılıkları DI ile bağla.

Yaklaşık	Odak	Çıktı
180–240 dk	Sorumluluk ayrımı	Katmanlı ürün API'si

Bugünün çalışma kuralı

Önce controller service ayrımını kur. DTO ve POST işlemini service bağlantısı çalıştıktan sonra ekle.

Bitiş ölçütü

- Controller ve service sorumluluklarını ayırabilmek
- DTO'nun neden kullanıldığını açıklamak
- Interface üzerinden constructor injection yapmak
- GET ve POST endpoint'lerini service aracılığıyla çalıştırmak

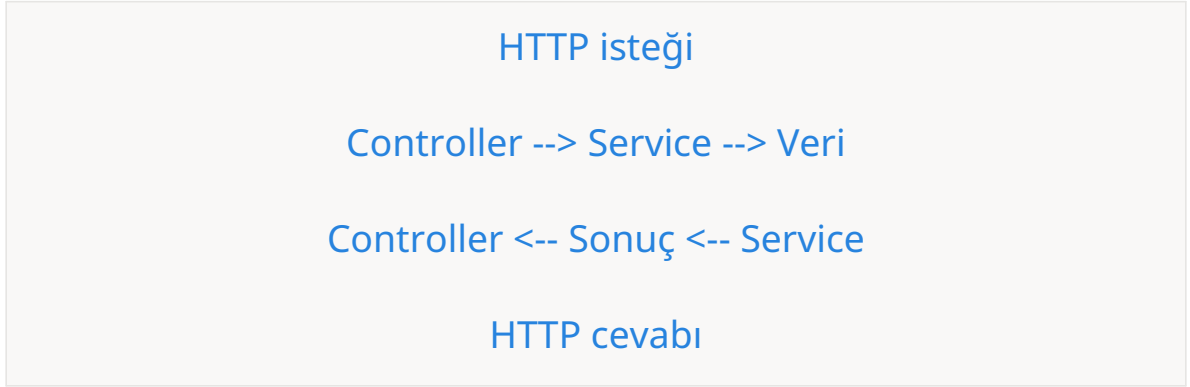
1. Controller neden bütün işi yapmamalı?

Controller'ın ana görevi HTTP isteğini almak ve uygun HTTP cevabını döndürmektir. Veri arama, doğrulama ve iş kuralları büyüdükçe ayrı bir service'e taşınır.

Kalabalık controller

```
[HttpGet("{id:int}")]
public IActionResult GetById(int id)
{
    // Listeyi ara
    // Kuralları kontrol et
    // Veriyi dönüştür
    // Log yaz
    // HTTP cevabını seç
}
```

Sorumluluk ayrımı



Controller	Service
Route ve HTTP metodu	Veri ve iş kuralları
Parametre/body alma	Arama, ekleme, hesaplama
Ok, NotFound, BadRequest	Uygulamaya ait sonuç üretme

Single Responsibility

Her class'ın değişmek için tek ana nedeni olması hedeflenir. Controller HTTP, service iş mantığı için değişir.

2. DTO nedir?

DTO (Data Transfer Object), API'ye giren veya API'den çıkan verinin şeklini tanımlar. Veri tabanı/entity modelini doğrudan dış dünyaya açmayı engeller.

Product modeli

```
public class Product
{
    public int Id { get; set; }
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
    public DateTime CreatedAt { get; set; }
}
```

Ürün oluşturma DTO'su

```
public class CreateProductDto
{
    public string Name { get; set; } = "";
    public decimal Price { get; set; }
}
```

İstemcinin gönderdiği JSON

```
{
  "name": "Klavye",
  "price": 850
}
```

Neden Id ve CreatedAt yok?

Bunları istemci değil, uygulama üretir. DTO yalnızca kullanıcının göndermesine izin verdiğimiz alanları içerir.

DTO avantajları

- Gereksiz veya hassas alanların dışarı açılmasını önler.
- Request ve response sözleşmesini açıklıştırır.
- İç model değişse bile API sözleşmesini korumayı kolaylaştırır.

3. Interface ile service sözleşmesi

Interface, service'in hangi işlemleri sunacağını belirtir; nasıl yaptığını belirtmez.

Services/IProductService.cs

```
public interface IProductService
{
    List<Product> GetAll();
    Product? GetById(int id);
    Product Create(CreateProductDto dto);
}
```

Metot	Anlam
GetAll	Bütün ürünleri döndürür.
GetById	Ürün veya bulunamazsa null döndürür.
Create	DTO'dan yeni Product üretilip döndürür.

Service iskeleti

```
public class ProductService : IProductService
{
    private readonly List<Product> _products = new()
    {
        new Product { Id = 1, Name = "Klavye", Price = 850 }
    };

    public List<Product> GetAll()
    {
        return /* ... */;
    }

    public Product? GetById(int id)
    {
        return _products.FirstOrDefault(
            product => /* id karşılaştırması */);
    }

    public Product Create(CreateProductDto dto)
    {
        // Doğrula, Id üret, listeye ekle ve ürünü döndür.
    }
}
```

Alt çizgi neden var?

_products, class alanı (field) olduğunu anlatan yaygın C# isimlendirme biçimidir.

4. Dependency Injection nedir?

Controller, ProductService nesnesini doğrudan new ile oluşturmak yerine ihtiyaç duyduğu sözleşmeyi ister. ASP.NET Core uygun nesneyi verir.

Sıkı bağımlılık

```
private readonly ProductService _service = new ProductService();
```

Constructor injection

```
private readonly IProductService _service;

public ProductsController(IProductService service)
{
    _service = service;
}
```

DI kaydı — Program.cs

```
builder.Services.AddSingleton<
    IProductService,
    ProductService>();
```

Parça	Anlamı
IProductService	Controller'ın istediği sözleşme
ProductService	Kullanılacak gerçek uygulama
AddSingleton	Uygulama boyunca aynı service nesnesi

Bu projede neden Singleton?

Veri bellekteki instance listesinde tutuluyor. Aynı service nesnesi korunursa eklenen ürünler sonraki isteklerde görülebilir. Veritabanı günlerinde Scoped kullanacağız.

5. Controller'a service enjekte etmek

Controller artık listeyi kendi içinde tutmaz; veriyi service'ten ister.

Constructor

```
private readonly IProductService _service;

public ProductsController(IProductService service)
{
    _service = service;
}
```

GET action'ları

```
[HttpGet]
public IActionResult GetAll()
{
    List<Product> products = _service.GetAll();
    return Ok(products);
}

[HttpGet("{id:int}")]
public IActionResult GetById(int id)
{
    Product? product = _service.GetById(id);

    if (product is null)
    {
        return NotFound(new
        {
            Message = "Ürün bulunamadı."
        });
    }

    return Ok(product);
}
```

Controller neyi bilmiyor?

Listenin nerede tutulduğunu ve ürünün nasıl arandığını bilmez. Yalnızca service sonucuna göre HTTP cevabı seçer.

İlk görev

Service'te GetAll ve GetById metotlarını tamamla; Program.cs kaydını ve controller constructor'ını kur. Sonra iki GET endpoint'ini test et.

6. POST ve CreateProductDto

POST isteğinin JSON body'si CreateProductDto nesnesine dönüştürülür. Buna deserialization denir.

Controller action iskeleti

```
[HttpPost]
public IActionResult Create(CreateProductDto dto)
{
    if (string.IsNullOrEmpty(dto.Name))
    {
        return BadRequest(new
        {
            Message = "Ürün adı boş olamaz."
        });
    }

    if (dto.Price <= 0)
    {
        return BadRequest(new
        {
            Message = "Fiyat pozitif olmalıdır."
        });
    }

    Product product = _service.Create(dto);

    return CreatedAtAction(
        nameof(GetById),
        new { id = product.Id },
        product);
}
```

CreatedAtAction ne döndürür?

- 201 Created durum kodu
- Oluşturulan Product JSON'u
- Location header içinde yeni kaynağın adresi

Service Create iskeleti

```
public Product Create(CreateProductDto dto)
{
    Product product = new Product
    {
        Id = /* yeni id */,
        Name = /* DTO adı */,
        Price = /* DTO fiyatı */,
        CreatedAt = DateTime.Now
    };

    // Listeye ekle.
    // Oluşturulan ürünü döndür.
}
```

Görev sırası

Önce GET bağlantısı çalışmalı. Ardından Create metodu ve POST action eklenmeli.

7. Test, proje yapısı ve Git

Önerilen yapı

```
Gun10-ServiceDtoDI/  
├── Controllers/ProductsController.cs  
├── Dtos/CreateProductDto.cs  
├── Models/Product.cs  
├── Services/IProductService.cs  
├── Services/ProductService.cs  
├── Program.cs  
└── Gun10-ServiceDtoDI.csproj
```

Test listesi

- GET /api/products 200 ve liste
- GET /api/products/1 200 ve ürün
- GET /api/products/99 404
- POST geçerli JSON 201
- POST boş ad veya negatif fiyat 400
- POST sonrası GET listesinde yeni ürün

Mülakat notları

- DTO, katmanlar arasında taşınacak veri şeklini tanımlar.
- Service iş mantığını controller'dan ayırır.
- DI bağımlılıkları dışarıdan sağlar; constructor injection yaygındır.
- Interface, controller'ı somut implementation'a daha az bağımlı yapar.

Git

```
cd "C:\Users\Administrator\.NetEgitim"  
git add .  
git commit -m "Gün 10 service DTO ve DI tamamlandı"  
git push
```

Gün 10 tamamlanma ölçütü

Controller'da liste kalmadıysa, GET service üzerinden çalışıyorsa ve POST 201 döndürüyorsa hedef tamamdır.